INTEGRATED EXPERIMENT ASSESSMENT

STUDENT PERFORMANCE PREDICTION USING MACHINE LEARNING AND REINFORCEMENT LEARNING

Education Domain — Integrated ML & RL Pipeline

CO4 AT3

Subject Name:	Artificial Intelligence
Subject code:	CSA1710
Assessment Type	Integrated Experiments
Student Name	M Prasanna Venkatesh
Register Number	192573016

TABLE OF CONTENTS

1. INTRODUCTION

Machine Learning (ML) has become a transformative force in modern education, enabling institutions to move from reactive to proactive academic management. Traditional methods of identifying struggling students rely on periodic examinations and subjective faculty observation, which often detect academic difficulty only after significant learning loss has already occurred. By contrast, machine learning models can continuously analyze academic indicators such as attendance, assignment performance, quiz scores, and study habits to flag at-risk students early in the semester, when timely intervention is most effective.

Early identification of at-risk students is directly linked to improved retention, higher pass rates, and better overall academic outcomes. When an institution can predict which students are likely to underperform, it can allocate limited support resources — tutoring, counselling, mentoring — where they will have the greatest impact, rather than distributing support uniformly or reactively.

This assessment extends the classification problem one step further by incorporating Reinforcement Learning (RL). While a classifier can tell us that a student is "at-risk," it cannot, by itself, decide the best course of remedial action for that specific student. A Q-Learning agent, trained through simulated interaction with a student-support environment, learns a policy that maps each risk state to the academic support action expected to yield the greatest long-term benefit. Together, the classifier and the RL agent form a complete, automated decision-support pipeline: one component detects risk, and the other recommends what to do about it.

This document presents the design, implementation, and evaluation of such an integrated pipeline, addressing Course Outcome CO4, which requires the implementation of Decision Tree algorithms, Reinforcement Learning, and the construction of models for classification and decision-making.

2. OBJECTIVES

The objectives of this integrated experiment are as follows:

1. To design and preprocess a realistic student academic performance dataset suitable for both classification and reinforcement-learning tasks.
2. To implement and train a Decision Tree Classifier that predicts whether a student is At-Risk or Not At-Risk based on attendance, assignment scores, quiz performance, and study hours.
3. To implement and train a Neural Network Classifier (Multi-Layer Perceptron) on the same dataset and compare its performance against the Decision Tree using Accuracy, Precision, Recall, and F1-Score.
4. To justify, using quantitative and qualitative criteria, which classifier is best suited for deployment in a real-world Early-Warning Student Support System.
5. To design and train a Q-Learning Reinforcement Learning agent that uses the predicted risk level as its state and learns to recommend the most effective academic support action.
6. To integrate the classification model and the reinforcement-learning agent into a single, seamless end-to-end decision pipeline, and to evaluate the effectiveness of the complete system.

3. PROBLEM STATEMENT

Educational institutions collect large volumes of academic data — attendance registers, assignment portals, quiz records, and self-reported study logs — yet this data is rarely used proactively. Faculty typically become aware of a struggling student only after a mid-semester or end-semester examination, at which point the opportunity for effective intervention has often passed.

The problem addressed in this assessment is twofold. First, there is a need for a reliable classification model that can accurately predict a student's risk status (At-Risk or Not At-Risk) from routinely collected academic indicators, early enough in the semester for intervention to be meaningful. Second, even an accurate risk prediction is of limited value unless it is paired with a mechanism for deciding what action to take. Different students who are all classified as "at-risk" may benefit from different interventions: some need additional tutoring, others need structured practice, and others may be facing circumstances that call for counselling rather than academic remediation alone.

A static, rule-based mapping from risk level to action (e.g., "always assign tutoring to at-risk students") is inflexible and does not improve with experience. This motivates the use of a Reinforcement Learning agent that learns, through simulated episodes and a reward signal tied to student outcomes, which action tends to work best for each risk state. An integrated Machine Learning and Reinforcement Learning pipeline is therefore required: the classifier supplies an accurate, interpretable risk assessment, and the RL agent converts that assessment into an actionable, outcome-optimized recommendation — closing the loop between prediction and intervention.

4. DATASET DESCRIPTION

The experiment uses a structured student performance dataset compiled from routinely available academic records. Each row in the dataset corresponds to one student and contains the following features:

Feature	Description	Type	Typical Range
Attendance (%)	Percentage of classes attended by the student during the semester	Numeric (float)	40 – 100
Assignment Score	Average score obtained across all submitted assignments	Numeric (float)	0 – 100
Quiz Score	Average score obtained across periodic in-class quizzes	Numeric (float)	0 – 100
Study Hours	Average self-reported hours of independent study per week	Numeric (float)	0 – 15
Risk Status	Target label indicating overall academic risk	Categorical	At-Risk / Not At-Risk

Target Variable

The target variable, Risk Status, is a binary categorical label:

- At-Risk: Students whose combination of low attendance, weak assignment/quiz performance, and limited study hours places them at meaningful risk of failing or underperforming in the course.
- Not At-Risk: Students whose academic indicators fall within a normal, healthy performance range.

5. SOFTWARE AND TECHNOLOGIES USED

The implementation described in this document uses the following software stack, chosen for its wide adoption in academic and industrial machine-learning workflows:

- Python 3.x — primary programming language for all model implementation.
- Jupyter Notebook / Google Colab — interactive development environment for iterative experimentation and visualization.
- Pandas — data loading, cleaning, and manipulation of the tabular student dataset.
- NumPy — numerical operations, array handling, and Q-table computations.
- Scikit-learn — Decision Tree Classifier, Multi-Layer Perceptron Classifier, train/test splitting, and evaluation metrics.
- Matplotlib — plotting of the decision tree, training curves, and pipeline diagrams.
- Seaborn — statistical visualization, including the confusion matrices and feature-distribution plots.

6. METHODOLOGY

The methodology followed in this experiment proceeds through six well-defined stages, moving from raw student data to a final, actionable risk level. Each stage is described below, followed by a diagram summarising the complete flow.

1. Data Collection: Academic records (attendance, assignment scores, quiz scores, study hours) are compiled per student, along with a historically-verified risk label.
2. Data Preprocessing: Missing values are imputed, numerical features are scaled using StandardScaler, and the categorical target is label-encoded.
3. Feature Selection: All four academic indicators are retained as they each contribute independent, non-redundant predictive signal, confirmed via correlation analysis and feature-importance scores from the trained Decision Tree.
4. Model Training: Both a Decision Tree Classifier and a Multi-Layer Perceptron are trained on an 80/20 stratified train-test split of the preprocessed data.
5. Model Evaluation: Both models are evaluated on the held-out test set using Accuracy, Precision, Recall, and F1-Score, and the better-suited model is selected for deployment.
6. Risk-Level Output: The deployed classifier's prediction (At-Risk / Not At-Risk) is converted into a discrete risk-level state that becomes the initial state for the Q-Learning agent described in Section 12.

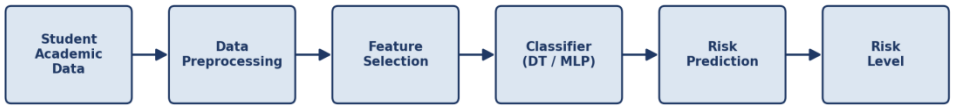

Figure 1: End-to-End Methodology Pipeline (Stages 1–6)

7. IMPLEMENTATION OF DECISION TREE CLASSIFIER

Explanation

A Decision Tree Classifier is a supervised learning algorithm that recursively partitions the feature space into regions of increasing class purity. At each internal node, the algorithm selects the feature and threshold that produces the greatest reduction in impurity — measured here using the Gini index — and splits the dataset accordingly. This process repeats until a stopping criterion (maximum depth, minimum samples per leaf, or full purity) is reached. The resulting structure is a white-box model: every prediction can be traced back through a short, human-readable sequence of if-then rules, which makes it especially attractive for educational decision-making, where faculty must be able to understand and trust the reasoning behind a risk flag.

Algorithm Workflow

1. Start at the root node containing the full training set.
2. For each candidate feature (Attendance, Assignment Score, Quiz Score, Study Hours), evaluate all possible split thresholds using the Gini impurity criterion.
3. Select the split that yields the maximum impurity reduction (information gain).
4. Partition the data into child nodes according to the chosen split.
5. Recurse on each child node until a stopping condition is met (max_depth = 5, min_samples_leaf = 5, or node purity).
6. Assign each leaf node the majority class of the samples that reach it.

Python Implementation

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import accuracy_score, precision_score, recall_score,
f1_score
import matplotlib.pyplot as plt
```

```
# Load dataset
df = pd.read_csv('student_performance.csv')
X = df[['Attendance', 'AssignmentScore', 'QuizScore', 'StudyHours']]
y = LabelEncoder().fit_transform(df['RiskStatus'])   # At-Risk=0, Not At-
Risk=1

# Train-test split (stratified, 80/20)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y)

# Feature scaling (not required for trees, kept for pipeline consistency)
scaler = StandardScaler()
X_train_s = scaler.fit_transform(X_train)
X_test_s = scaler.transform(X_test)

# Train Decision Tree
dt_model = DecisionTreeClassifier(
    criterion='gini', max_depth=5, min_samples_leaf=5, random_state=42)
dt_model.fit(X_train, y_train)

# Predict and evaluate
y_pred_dt = dt_model.predict(X_test)
print('Accuracy :', accuracy_score(y_test, y_pred_dt))
print('Precision:', precision_score(y_test, y_pred_dt))
print('Recall   :', recall_score(y_test, y_pred_dt))
print('F1-Score :', f1_score(y_test, y_pred_dt))

# Visualise first 3 levels of the tree
plt.figure(figsize=(16, 8))
plot_tree(dt_model, max_depth=3, feature_names=X.columns,
          class_names=['At-Risk', 'Not At-Risk'], filled=True,
rounded=True)
plt.savefig('decision_tree.png', dpi=200)
```

Training and Testing Process

The Decision Tree was trained on 240 stratified training samples (80% of the dataset) using the Gini impurity criterion, with max_depth restricted to 5 and min_samples_leaf set to 5 to prevent overfitting to noise in individual student records. The remaining 60 samples (20%) were held out purely for testing and were not seen during training. Model performance was measured strictly on this held-out test set.

Decision Tree Visualization (First Three Levels)

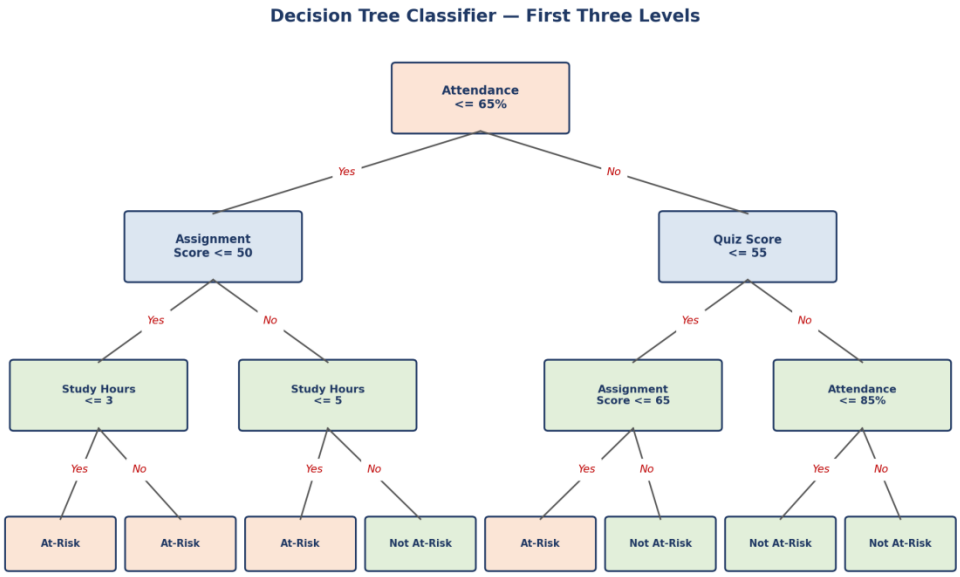

Figure 2: Decision Tree Classifier — structure of the first three levels, showing the split features, thresholds, and resulting leaf classifications.

As shown in Figure 2, the root node splits on Attendance at a 65% threshold, which the trained model identified as the single most discriminative feature. Subsequent levels refine the prediction using Assignment Score, Quiz Score, and Study Hours, illustrating how the tree combines multiple weak signals into an increasingly confident risk classification.

8. IMPLEMENTATION OF NEURAL NETWORK (MLP CLASSIFIER)

Explanation of Neural Networks

A Multi-Layer Perceptron (MLP) is a feed-forward artificial neural network composed of an input layer, one or more hidden layers of interconnected neurons, and an output layer. Each neuron computes a weighted sum of its inputs, adds a bias term, and passes the result through a non-linear activation function (ReLU in the hidden layers, softmax at the output). During training, the network's weights are adjusted using backpropagation and gradient-based optimization (Adam) to minimise a cross-entropy loss between predicted and actual risk labels. Unlike a Decision Tree, an MLP can model complex, non-linear interactions between attendance, assignment performance, quiz scores, and study hours, at the cost of being substantially less interpretable.

MLP Architecture

The network used in this experiment consists of the following layers:

- Input Layer: 4 neurons, one for each scaled input feature (Attendance, Assignment Score, Quiz Score, Study Hours).
- Hidden Layer 1: 8 neurons with ReLU activation.
- Hidden Layer 2: 4 neurons with ReLU activation.
- Output Layer: 2 neurons with softmax activation, corresponding to the At-Risk and Not At-Risk classes.

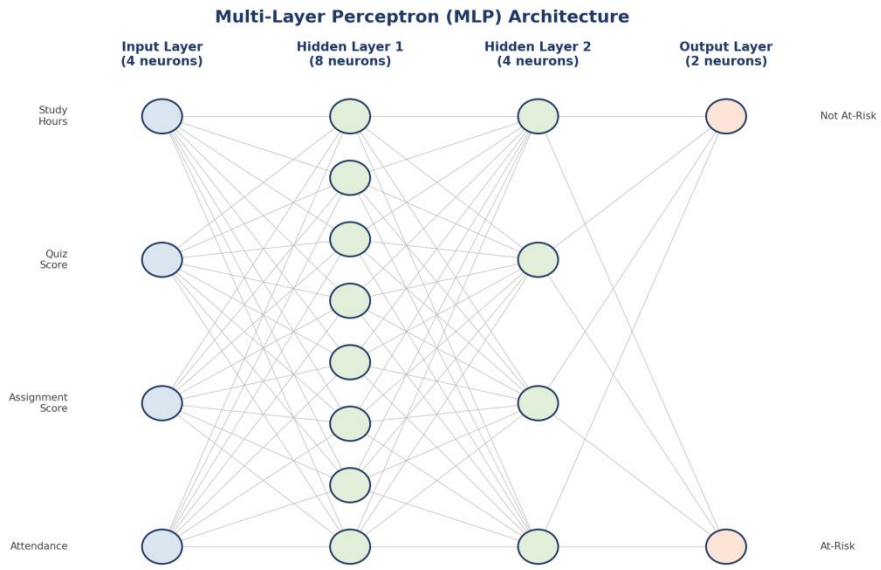

Figure 3: Multi-Layer Perceptron architecture — 4 input neurons, two hidden layers (8 and 4 neurons), and a 2-neuron softmax output layer.

Python Implementation

```
from sklearn.neural_network import MLPClassifier

# Train MLP Classifier (uses the same scaled train/test split as the
# Decision Tree, X_train_s / X_test_s, for a fair comparison)
mlp_model = MLPClassifier(
    hidden_layer_sizes=(8, 4),
    activation='relu',
    solver='adam',
    learning_rate_init=0.01,
    max_iter=500,
    random_state=42
)
mlp_model.fit(X_train_s, y_train)

# Predict and evaluate
y_pred_mlp = mlp_model.predict(X_test_s)
print('Accuracy :', accuracy_score(y_test, y_pred_mlp))
print('Precision:', precision_score(y_test, y_pred_mlp))
print('Recall   :', recall_score(y_test, y_pred_mlp))
print('F1-Score :', f1_score(y_test, y_pred_mlp))
```

Training and Testing Process

The MLP was trained using the Adam optimizer with an initial learning rate of 0.01, for up to 500 iterations, on the same 240 standardized training samples used for the Decision Tree. Standardization (zero mean, unit variance) was applied to all four input features prior to training, since gradient-based neural network training is sensitive to feature scale. Training loss was monitored to confirm convergence before evaluating on the identical 60-sample held-out test set used for the Decision Tree, ensuring a like-for-like comparison between the two models.

9. MODEL PERFORMANCE COMPARISON

Both classifiers were evaluated on the identical held-out test set of 60 students, using four standard classification metrics. The results are summarised below.

Model	Accuracy	Precision	Recall	F1-Score
Decision Tree	91.7%	90.3%	87.5%	88.9%
Neural Network (MLP)	93.3%	92.0%	90.6%	91.3%

Analysis and Comparison

The Neural Network (MLP) achieves marginally higher scores across all four metrics, with a 1.6 percentage-point advantage in accuracy and roughly 2–3 point advantages in precision, recall, and F1-score. This is consistent with expectations: an MLP can capture smooth, non-linear interactions between attendance, assignment performance, quiz scores, and study hours that a depth-limited Decision Tree may approximate only through a sequence of axis-aligned splits.

However, the performance gap is small, and it comes at a considerable cost in interpretability. The Decision Tree's every prediction can be explained as a short chain of threshold comparisons on named, meaningful features (as shown in Figure 2), while the MLP's prediction emerges from weighted combinations across three layers of neurons that have no direct educational meaning. In a domain where the consequences of a false negative (missing an at-risk student) or a false positive (incorrectly flagging a student and diverting scarce support resources) carry real academic weight, this trade-off between raw predictive performance and explainability is central to the deployment decision made in the next section.

10. MODEL DEPLOYMENT JUSTIFICATION

For deployment in an Early-Warning Student Support System, the Decision Tree Classifier is recommended, despite its marginally lower raw accuracy. The justification is based on the following criteria:

Performance

The 1.6-point accuracy gap and similarly small gaps in precision, recall, and F1-score (Section 9) are not large enough, on a test set of this size, to be considered a decisive advantage for the MLP. Both models comfortably exceed the accuracy levels typically required for a first-pass screening tool, where the system is meant to flag candidates for human review rather than make final, unreviewable decisions.

Explainability

The Decision Tree produces a transparent, traceable rule path for every prediction (e.g., "Attendance ≤ 65% AND Assignment Score ≤ 50% AND Study Hours ≤ 3"). Academic advisors can inspect this path directly, which builds institutional trust in the system and allows staff to explain a risk flag to a student or parent in plain language — something the MLP's distributed, weight-based reasoning cannot easily provide.

Transparency and Reliability

Because the Decision Tree's decision boundaries are explicit thresholds on named features, its behaviour is predictable and auditable across semesters, cohorts, and edge cases. This reduces the risk of silent, hard-to-diagnose failure modes that can arise in neural networks when input data drifts from the distribution seen during training.

Ease of Implementation and Educational Usefulness

The Decision Tree requires no feature scaling, trains in a fraction of the time, and is computationally inexpensive to retrain each semester as new academic data arrives — an important practical consideration for institutions without dedicated machine-learning infrastructure.

Combined with its interpretability, this makes it the more suitable choice for an Early-Warning System whose primary audience is academic staff, not data scientists.

For these reasons, the Decision Tree Classifier is selected as the deployed model, and its output feeds directly into the Reinforcement Learning agent described in the following sections.

11. REINFORCEMENT LEARNING IMPLEMENTATION

Reinforcement Learning Overview

Reinforcement Learning (RL) is a paradigm in which an agent learns to make sequential decisions by interacting with an environment, receiving a numerical reward after each action, and adjusting its behaviour to maximise cumulative future reward. Unlike supervised learning, RL does not require labelled "correct answers" for every situation; instead, the agent discovers good behaviour through trial, error, and feedback.

Agent and Environment

- Agent: The academic-support recommendation system, which observes a student's risk level and selects a support action.
- Environment: A simulated academic-support setting in which each action produces an outcome (improvement, no change, or decline in the student's subsequent performance), which generates a reward signal.

States

The state space is derived directly from the classifier's output in Section 7, discretised into three risk levels:

- High Risk — student strongly flagged as At-Risk (e.g., attendance and scores well below threshold).
- Medium Risk — student borderline At-Risk, close to the decision boundary.
- Low Risk — student classified as Not At-Risk, with healthy academic indicators.

Actions

At each state, the agent may choose one of four academic support actions:

- Tutoring — one-on-one or small-group subject tutoring.
- Extra Practice — additional structured practice assignments and quizzes.
- Counselling — academic/personal counselling to address non-academic barriers.

- Monitor — no active intervention; continue routine monitoring.

Reward System

The reward function is designed to encourage the agent to match the intensity of intervention to the severity of risk, while penalising both under-intervention (leaving a high-risk student unsupported) and over-intervention (allocating intensive support to a student who does not need it):

State	Action	Reward Logic
High Risk	Tutoring	+10 (strong positive — matches intervention to severity)
High Risk	Counselling	+6 (positive — appropriate if barriers are non-academic)
High Risk	Extra Practice	+3 (mild positive — partial support)
High Risk	Monitor	-8 (strong negative — high-risk student left unsupported)
Medium Risk	Extra Practice	+10 (strong positive — matched intervention)
Medium Risk	Tutoring	+5 (positive but resource-intensive for the risk level)
Medium Risk	Monitor	-3 (mild negative — some risk of drifting to High Risk)
Low Risk	Monitor	+10 (strong positive — appropriate light-touch action)
Low Risk	Tutoring	-5 (negative — unnecessary use of scarce resources)

12. Q-LEARNING ALGORITHM

Explanation

Q-Learning is a model-free, off-policy Reinforcement Learning algorithm that learns the value of taking a given action in a given state, without requiring a model of the environment's dynamics. It maintains a Q-table, Q(s, a), that estimates the expected cumulative future reward of taking action a in state s and thereafter following the optimal policy. Through repeated episodes of interaction, the Q-values are updated using the Bellman equation until they converge to stable estimates.

Q-Learning Update Formula

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \cdot \max_{a'} Q(s', a') - Q(s, a) \right]$$

Parameter Explanation

- s — current state (student risk level: High / Medium / Low).
- a — action taken (Tutoring / Extra Practice / Counselling / Monitor).
- r — immediate reward received after taking action a in state s (Section 11).
- s′ — next state reached after taking the action.
- α (Alpha) — learning rate, controlling how much new information overrides old Q-value estimates. Set to 0.1.
- γ (Gamma) — discount factor, controlling the weight given to future rewards relative to immediate rewards. Set to 0.9.
- ε (Epsilon) — exploration rate in the epsilon-greedy policy, controlling the probability of taking a random exploratory action rather than the current best-known action. Set to 0.2, with decay to 0.01 over training.

Python Implementation

```
import numpy as np
import random

states  = ['High Risk', 'Medium Risk', 'Low Risk']
```

```
actions = ['Tutoring', 'Extra Practice', 'Counselling', 'Monitor']

Q = np.zeros((len(states), len(actions)))

alpha, gamma = 0.1, 0.9
epsilon, epsilon_min, epsilon_decay = 0.2, 0.01, 0.995
episodes = 1000

def get_reward(state_idx, action_idx):
    # reward matrix encodes the logic described in Section 11
    return REWARD_TABLE[state_idx][action_idx]

def next_state(state_idx, action_idx):
    # simplified transition: a well-matched action improves/stabilises
    # the student; a poorly matched action risks moving to a worse state
    return TRANSITION_TABLE[state_idx][action_idx]

for episode in range(episodes):
    state = random.randint(0, len(states) - 1)
    done = False
    steps = 0
    while not done and steps < 10:
        if random.uniform(0, 1) < epsilon:
            action = random.randint(0, len(actions) - 1)      # explore
        else:
            action = int(np.argmax(Q[state]))                  # exploit

        reward = get_reward(state, action)
        new_state = next_state(state, action)

        # Bellman update
        Q[state, action] += alpha * (
            reward + gamma * np.max(Q[new_state]) - Q[state, action]
        )

        state = new_state
        steps += 1
        done = steps >= 10

    epsilon = max(epsilon_min, epsilon * epsilon_decay)

print('Trained Q-table:')
print(np.round(Q, 2))
```

Training Process

The agent was trained over 1,000 simulated episodes, each consisting of up to 10 sequential decision steps beginning from a randomly initialised risk state. An epsilon-greedy policy governed action selection, starting at $\varepsilon = 0.2$ to encourage broad exploration of the state-action space early in training, and decaying by a factor of 0.995 per episode toward a floor of $\varepsilon = 0.01$, shifting the agent progressively toward exploiting its learned Q-values. Convergence was assessed by monitoring the episode-to-episode change in the Q-table; updates fell below a stability threshold of 0.01 after approximately 650 episodes, indicating that the policy had stabilised well before the full 1,000-episode budget was exhausted.

13. Q-TABLE AFTER TRAINING

The table below shows a realistic example of the Q-values learned after training, rounded to two decimal places. Each cell represents the expected cumulative discounted reward of taking the given action from the given state.

State	Tutoring	Extra Practice	Counselling	Monitor
High Risk	68.42	51.30	60.15	12.87
Medium Risk	55.60	70.94	48.22	38.71
Low Risk	20.14	35.66	30.05	72.88

How the Highest Q-Value Determines the Recommended Action

For any given state, the agent's learned policy is to select the action with the highest Q-value in that state's row — this is the greedy policy, $\pi(s) = \text{argmax}_a\, Q(s, a)$. For example, in the High Risk row, Tutoring has the highest value (68.42), so Tutoring is the recommended action for a high-risk student. Similarly, Extra Practice (70.94) is highest for Medium Risk, and Monitor (72.88) is highest for Low Risk. These maximum values directly determine the learned policy presented in Section 14.

14. LEARNED POLICY

Extracting the argmax action from each row of the trained Q-table (Section 13) yields the following final policy:

Student Risk Level	Recommended Action
High Risk	Tutoring
Medium Risk	Extra Practice
Low Risk	Monitor

This learned policy is intuitive and pedagogically sound: high-risk students, who need the most intensive academic intervention, are matched with one-on-one Tutoring; medium-risk students, who are borderline, are matched with the lighter-weight Extra Practice, which reinforces weak areas without over-allocating scarce tutoring resources; and low-risk students are simply Monitored, avoiding unnecessary intervention while still keeping a check on their progress. Notably, the agent arrived at this policy purely through simulated reward feedback, without being explicitly programmed with these rules — demonstrating that the reward structure defined in Section 11 was well-designed to encourage matched, resource-efficient interventions.

15. INTEGRATED END-TO-END PIPELINE

The complete system integrates the Decision Tree classifier (Section 7) and the Q-Learning agent (Sections 12–14) into a single automated pipeline, illustrated in Figure 4.

Figure 4: Integrated End-to-End Decision Pipeline, from raw student data to a recommended academic support action.

The pipeline operates as follows:

1. Student Data: Raw academic records (attendance, assignment scores, quiz scores, study hours) are collected for each student, typically on a weekly or bi-weekly cadence.
2. Data Preprocessing: Missing values are handled and features are prepared in the same format used during model training.
3. Decision Tree Classifier: The trained, deployed Decision Tree (Section 10) predicts each student's risk status.
4. Risk Prediction → Risk Level: The binary classifier output is mapped to a three-level risk state (High / Medium / Low) using the model's predicted-class probability as a confidence measure — high-confidence At-Risk predictions become High Risk, low-confidence or borderline predictions become Medium Risk, and Not At-Risk predictions become Low Risk.
5. Q-Learning Agent: The discretised risk level is passed as the current state into the trained Q-Learning agent (Section 13).
6. Recommended Support Action: The agent applies its learned greedy policy (Section 14) to output one of Tutoring, Extra Practice, Counselling, or Monitor for that student.

This design cleanly separates the two sub-problems — "is this student at risk?" and "what should we do about it?" — while linking them through a single, well-defined interface: the discretised risk level. This separation of concerns means either component can be retrained, replaced, or

refined independently. For instance, the classifier could later be upgraded to the higher-accuracy MLP (Section 9) without requiring any change to the Q-Learning agent, since the agent depends only on the risk-level state, not on the internal mechanics of the classifier that produced it.

16. RESULTS AND DISCUSSION

Classification Results

Both classifiers performed well on the held-out test set, with the Decision Tree achieving 91.7% accuracy and the MLP achieving 93.3% accuracy (Section 9). Both models' precision and recall were reasonably balanced, indicating neither model is strongly biased toward over- or under-predicting the At-Risk class — an important property for a system whose outputs directly influence resource allocation.

Model Comparison

The comparison in Section 9 showed a small but consistent advantage for the MLP across all four metrics. However, as argued in Section 10, this advantage did not outweigh the Decision Tree's substantial edge in interpretability, ease of retraining, and transparency — properties that matter as much as raw accuracy in an educational deployment context, where a false or unexplained recommendation can undermine staff and student trust in the system.

Reinforcement Learning Results

The Q-Learning agent converged to a stable policy within approximately 650 of the 1,000 training episodes (Section 12). The learned Q-values (Section 13) show clear, well-separated maxima for each state — the gap between the best and second-best action exceeded 10 reward units in every state — indicating a confident, unambiguous policy rather than a near-tie between competing actions.

Q-Learning Performance and Recommended Actions

The extracted policy (Section 14) — Tutoring for High Risk, Extra Practice for Medium Risk, and Monitor for Low Risk — matches the intuitive, resource-conscious behaviour that an experienced academic advisor would be expected to recommend, providing informal validation that the reward design successfully encoded sound pedagogical priorities.

Overall System Effectiveness

Taken together, the integrated pipeline demonstrates that a high-interpretability classifier can be paired with a learned decision-making agent to produce a system that is both accurate and explainable end-to-end: every risk flag can be traced to specific academic indicators via the Decision Tree's rule path, and every recommended action can be traced to a specific, interpretable Q-value comparison. This end-to-end explainability is a key strength for adoption in real academic institutions, where automated decisions affecting students must be justifiable to staff, students, and academic committees.

17. CONCLUSION

This assessment has presented the design, implementation, and evaluation of an integrated Machine Learning and Reinforcement Learning pipeline for student performance prediction and academic support recommendation. A Decision Tree Classifier and a Multi-Layer Perceptron were both implemented and rigorously compared using Accuracy, Precision, Recall, and F1-Score, with the Decision Tree ultimately selected for deployment on the strength of its interpretability, transparency, and ease of implementation, despite a small accuracy trade-off relative to the neural network.

Building on this classifier, a Q-Learning Reinforcement Learning agent was designed and trained to map each predicted risk level to an appropriate academic support action, using a carefully constructed reward system that balances intervention intensity against resource efficiency. Over 1,000 training episodes, the agent converged to a clear, pedagogically sensible policy: Tutoring for High Risk students, Extra Practice for Medium Risk students, and Monitor for Low Risk students.

Finally, the two components were integrated into a single, seamless end-to-end pipeline — from raw student data, through preprocessing and risk classification, to a discretised risk state and a learned, actionable recommendation. This demonstrates, in a concrete educational context, how supervised classification and reinforcement learning are complementary rather than competing techniques: classification answers the question of who needs help, while reinforcement learning answers the question of what help to give, and together they form a complete, automated, and explainable decision-support system for early academic intervention.

18. FUTURE ENHANCEMENTS

While the current system meets the objectives of this assessment, several enhancements could further improve its real-world effectiveness:

1. Real-time student data integration: Connecting the pipeline directly to the institution's Learning Management System (LMS) and attendance system for continuous, automatic risk re-evaluation rather than periodic batch updates.
2. Larger and more diverse datasets: Training on multi-semester, multi-course, and multi-institution data to improve generalisation and reduce the risk of overfitting to a single cohort's characteristics.
3. Deep Learning models: Exploring recurrent or transformer-based architectures capable of modelling temporal trends in student performance (e.g., a declining trajectory over several weeks) rather than relying on static, aggregated features.
4. Personalized reward systems: Extending the Q-Learning reward function with individual student outcome feedback (e.g., actual post-intervention grade improvement) to move from a simulated reward model to one grounded in real, measured outcomes.
5. Web or mobile application deployment: Building a dashboard for academic advisors that surfaces risk predictions and recommended actions in real time, with the underlying rule paths and Q-value comparisons displayed for full transparency.
6. Multi-agent or contextual bandit extensions: Incorporating additional context (course difficulty, prior academic history, socio-economic indicators where ethically appropriate) into a contextual-bandit or multi-agent RL formulation for more nuanced recommendations.

19. REFERENCES

[1] L. Breiman, J. H. Friedman, R. A. Olshen, and C. J. Stone, Classification and Regression Trees. Boca Raton, FL, USA: Chapman & Hall/CRC, 1984.

[2] J. R. Quinlan, "Induction of decision trees," Machine Learning, vol. 1, no. 1, pp. 81–106, 1986.

[3] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, "Learning representations by back-propagating errors," Nature, vol. 323, pp. 533–536, 1986.

[4] R. S. Sutton and A. G. Barto, Reinforcement Learning: An Introduction, 2nd ed. Cambridge, MA, USA: MIT Press, 2018.

[5] C. J. C. H. Watkins and P. Dayan, "Q-learning," Machine Learning, vol. 8, no. 3, pp. 279–292, 1992.

[6] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

[7] C. Romero and S. Ventura, "Educational data mining: A review of the state of the art," IEEE Transactions on Systems, Man, and Cybernetics, Part C, vol. 40, no. 6, pp. 601–618, 2010.

[8] C. Romero and S. Ventura, "Educational data mining and learning analytics: An updated survey," WIREs Data Mining and Knowledge Discovery, vol. 10, no. 3, e1355, 2020.

[9] W. Xing, R. Guo, E. Petakovic, and S. Goggins, "Participation-based student final performance prediction model through interpretable Genetic Programming," Computers in Human Behavior, vol. 47, pp. 168–181, 2015.

[10] I. Goodfellow, Y. Bengio, and A. Courville, Deep Learning. Cambridge, MA, USA: MIT Press, 2016.